

Flutter Developer.

From zero to a polished Flutter app, in five days.

"If it doesn't run on a real phone, it didn't happen."

A complete, paste-ready Antigravity build guide for the Mobile track of the Bulao project — AI Seekho 2026.

Every prompt is self-contained. Every file path, dependency, schema, and convention is spelled out. Antigravity will not need to guess.

How to use this document:

1. Read 'Your role' and 'System context' first (sections 01-02).
2. On each build day, paste the day's prompt into Antigravity.
3. Verify with the day's checklist before moving on.
4. Reference the appendices when Antigravity asks for clarification.

01 · YOUR ROLE

You own mobile.

You are the **Flutter Developer** on the Bulao build team. There are four teammates total — Product & Urdu, Mobile, Backend, Infrastructure & Trace. This document is yours alone. Each teammate has their own pack. Do not try to do the others' work; coordinate handoffs through WhatsApp + a shared Drive.

What you own

- The entire Flutter codebase (mobile/ directory)
- Press-and-hold voice capture UI
- Six AgentCard widgets with cinematic reveal animations
- Pricing Card with expandable line-items
- Service-quality lifecycle UI (en_route → completed)
- Dispute screen + resolution chat UI
- Signed Android APK delivered to Infra by Day 5

What you do *not* own

- The backend Python code or system prompts
- Mock provider data generation
- GCP / Cloud Run / Firestore

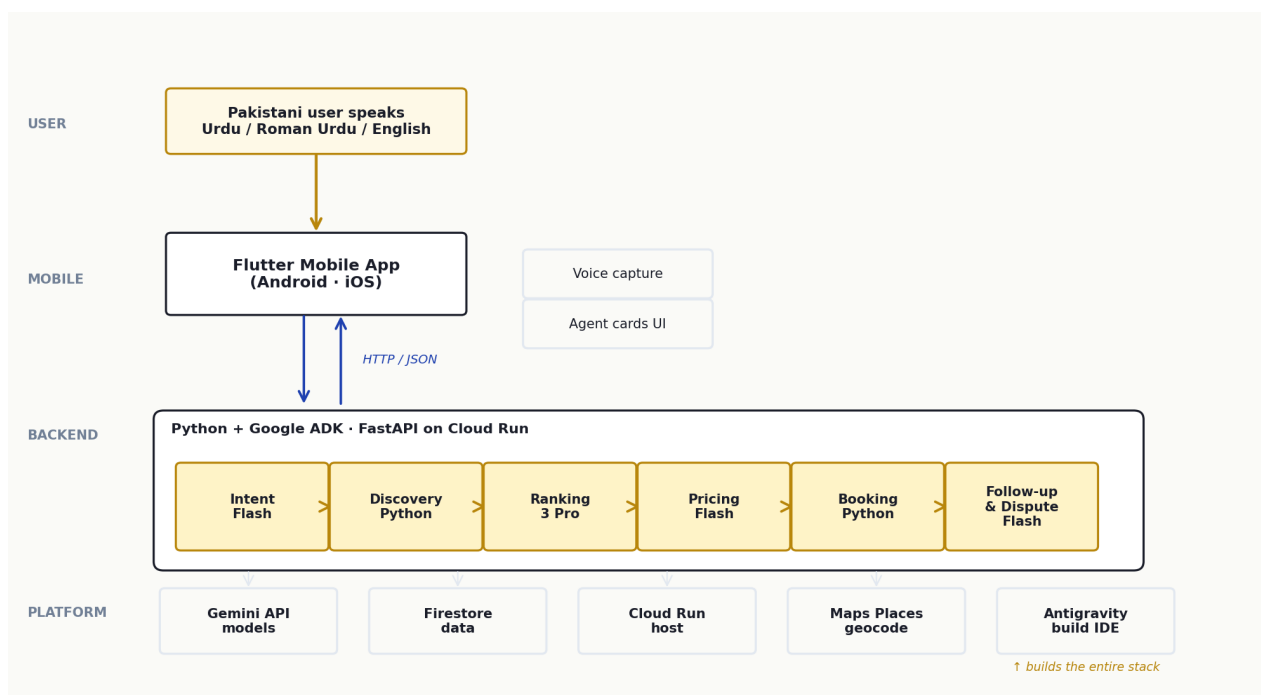
If you find yourself doing work outside the 'what you own' list, stop and ask in the team WhatsApp. Five days is short. Stay in your lane and move fast.

02 · SYSTEM CONTEXT

What Bulao is, end to end.

Bulao is a voice-first, Urdu-native AI service orchestrator for Pakistan's informal economy — plumbers, electricians, AC technicians, beauticians, tutors. A user presses a big button, speaks in Urdu / Roman Urdu / English, and within ninety seconds watches **six AI agents** extract their intent, find providers, rank them, produce a transparent price quote, book a slot, and follow up through completion — including fair dispute resolution when something goes wrong.

We are building this for the AI Seekho 2026 Antigravity Hackathon (Challenge 2). Five days, four teammates, one demo on May 20. Regional finals May 25-26, finals June 7 in Islamabad. Prize pool PKR 2.5M.



Three layers: a Flutter mobile app captures voice and renders six AgentCards; a Python backend on Cloud Run runs the six-agent pipeline using Google ADK and Gemini; Antigravity is the IDE that builds the stack — not part of the runtime, but the source of the 25%-weighted Agent Trace deliverable.

The six agents

Every teammate must know what each agent does, what it takes in, and what it produces. This is the shared mental model.



Your relationship to the six agents

You consume the agents' outputs and render them as cinematic UI. The `/orchestrate` endpoint returns a single JSON with six agent outputs; your app reveals them one by one as animated AgentCards. Each card has a specific visual treatment — the AgentCard widget you'll build handles pending / active / done states. You also render the Pricing Card (line-items with Urdu labels), the service-quality lifecycle screen (provider en route → arrived → in progress → complete), and the dispute screen when rating < 3.

Key handoffs you must respect

FROM	Day	TO	Deliverable
Infra	1 evening	You	Cloud Run backend URL (stub responses OK)
Backend	2 evening	You	Real <code>/orchestrate</code> returns valid Intent + Discovery
Backend	3 evening	You	All 6 agent outputs in <code>/orchestrate</code> response
You	3 evening	All	First end-to-end demo on a real device
You	5 morning	Infra	Signed APK + download link
You	5 afternoon	Infra	Demo screen recordings (3 angles)

03 · DAY 1 · BEFORE MAY 15

Day 0 (May 14) · setup

Install Flutter, Android Studio, the emulator, and Antigravity. Confirm the backend stub is reachable at the emulator loopback. No Antigravity prompts today.

Setup · Flutter + Android environment

These are not Antigravity prompts. Do them by hand before May 15. A broken Flutter environment burns Day 1. If you get stuck, post in WhatsApp before midnight — do not start Day 1 debugging dependencies.

PROMPT · LOCAL ENVIRONMENT CHECKLIST (DO YOURSELF)

1. Install Flutter SDK (stable channel, 3.x):
`https://docs.flutter.dev/get-started/install`
`flutter --version` # must show 3.x.x
2. Install Android Studio (for the emulator + SDK tools).
Open SDK Manager -> SDK Tools -> install:
Android SDK Build-Tools 34
Android SDK Platform-Tools
Android Emulator
Create a Virtual Device: Pixel 7 Pro API 34.
Launch the emulator and confirm it boots to the home screen.
3. Install VS Code with these extensions:
 - Flutter (`dart-code.flutter`)
 - Dart (`dart-code.dart-code`)
 - Pubspec Assist
4. Install Google Antigravity. Settings -> Agent panel:
Mode = Agent-assisted development
Artifact Review Policy = Asks for Review
Terminal Command Auto Execution = Request Review
Default model = Gemini 3 Pro
5. Clone the team repo (Infra will share URL).
`git clone`
`cd bulao/mobile`
`flutter pub get`
`flutter run -d emulator` # should show a white screen (app not built yet)
6. Add these entries to your local `.env` (NOT committed):
`BACKEND_BASE_URL=http://10.0.2.2:8080` # emulator loopback to laptop
`DEMO_MODE=false`
7. Confirm connectivity to the stub backend (Backend teammate runs locally):
`curl http://10.0.2.2:8080/health`
should return: `{"status": "ok", "version": "stub"}`

Day 1 starts immediately at 9am. Environment must be ready before then.

End-of-day verification

- flutter --version shows 3.x.x on stable channel.
- Android emulator launches and shows the home screen without errors.
- flutter doctor shows no errors (warnings OK).
- Antigravity launches and you can spawn a new Manager View workspace.
- Repo cloned; mobile/ directory exists.
- Backend stub confirmed reachable at <http://10.0.2.2:8080/health>.

04 · DAY 2 · MAY 15

Day 1 · scaffold the Flutter app

By 6pm tonight the app runs on the emulator with HomeScreen, MicButton, and all six AgentCards animating through stub data. The visual treatment is production-quality from Day 1 because judges will see exactly this in the demo.

Morning · project scaffold and app theme

Antigravity will scaffold the entire project structure, create the pubspec.yaml with all dependencies, set up the design token theme, and produce a HomeScreen with the press-and-hold mic button. By noon the app runs on the emulator with a realistic-looking home screen, even though no API calls are wired yet.

PROMPT · SCAFFOLD FLUTTER APP SKELETON (ANTIGRAVITY)

Create a production-quality Flutter app skeleton at mobile/ for the Bulao project. This is a voice-first AI service booking app for Pakistan.

PROJECT STRUCTURE TO CREATE:

```
mobile/  
lib/  
  main.dart  
  theme/app_theme.dart  
  models/  
    intent_model.dart  
    orchestrate_response.dart  
    booking_model.dart  
    pricing_model.dart  
  services/  
    api_service.dart  
  screens/  
    home_screen.dart  
    processing_screen.dart  
    pricing_screen.dart  
    booking_success_screen.dart  
    lifecycle_screen.dart  
    dispute_screen.dart  
  widgets/  
    agent_card.dart  
    mic_button.dart  
    pricing_card.dart  
    lifecycle_tracker.dart  
    dispute_widget.dart
```


PUBSPEC.YAML — include these dependencies:

```
flutter_sound: ^9.2.13 # audio recording
http: ^1.2.0 # REST calls
provider: ^6.1.2 # state management
google_fonts: ^6.1.0 # DM Sans for body text
animated_text_kit: ^4.2.2 # TypewriterAnimatedText for agent card reveals
lottie: ^3.1.0 # loading animations on agent cards
path_provider: ^2.1.2 # temp file for audio
permission_handler: ^11.3.0 # mic permission
intl: ^0.19.0 # date/time formatting
flutter_dotenv: ^5.1.0 # read .env for backend URL
```

APP THEME (mobile/lib/theme/app_theme.dart):

```
INK = Color(0xFF1A1D29)
GOLD = Color(0xFFB8860B)
GOLD_LIGHT = Color(0xFFFF5EDD0)
CREAM = Color(0xFFFEF9E7)
PAPER = Color(0xFFFFFAFAF7)
SLATE = Color(0xFF6B7280)
WHITE = Color(0xFFFFFFFF)
CRIMSON = Color(0xFF991B1B)
COBALT = Color(0xFF1E40AF)
FOREST = Color(0xFF166534)
```

Font: DM Sans for all text. Bold 28 for large headings, Bold 20 for section heads, Regular 15 for body.

Background: PAPER.

Accent: GOLD.

HOME SCREEN (mobile/lib/screens/home_screen.dart):

- AppBar: "Bulao" in Helvetica Bold gold on INK background.
- Body: centered vertically.
- Top: greeting text "Kya kaam hai aaj?" in SLATE.
- Center: MicButton widget (see below).
- Bottom: subtitle "Tap and hold to speak" in SLATE 13pt.

MIC BUTTON WIDGET (mobile/lib/widgets/mic_button.dart):

- Outer circle: 140px diameter, GOLD border 2px.
- Inner circle: 100px, GOLD fill.
- Mic icon: white, 42px.
- On long-press start: scale to 1.15, pulse animation (300ms loop), border becomes 4px.
- On long-press end: scale back, navigate to ProcessingScreen with the recorded audio.
- The GestureDetector wraps an AnimatedContainer with curve Curves.easeInOut, duration 200ms.

PROCESSING SCREEN STUB (mobile/lib/screens/processing_screen.dart):

- Takes a String parameter: userText (transcribed).
- AppBar: back arrow, title "Processing..."
- Body: ListView of 6 AgentCard widgets in a column.
- All 6 AgentCards start in "pending" state.
- DO NOT wire to any API yet — just the UI shell.

AGENT CARD WIDGET STUB (mobile/lib/widgets/agent_card.dart):

- Accepts: agentIndex (int), agentName (String), agentStatus (AgentStatus enum: pending/active/done), content (String?).
- pending: muted background (PAPER + 50% opacity), agent name in SLATE, no content.
- active: white background, 2px GOLD left border, agent name in INK Bold, content area shows a Lottie spinner.
- done: white background, gold checkmark top-right, content area shows the reveal text.

After scaffolding, run:

flutter analyze # should show zero errors

flutter test # should pass the basic widget test

flutter run -d emulator # HomeScreen should render correctly

The app should render the HomeScreen with the press-and-hold MicButton by end of this prompt. Backend is not wired yet — that's tomorrow.

Afternoon · ProcessingScreen + 6 stub AgentCards

Now build the full ProcessingScreen with all six AgentCards — still stub data, but the complete visual treatment: cards reveal one by one on a timer, the TypewriterAnimatedText effect types out the content, and the done-state checkmark appears. Judges will see exactly this screen during the demo; it must look finished from Day 1 evening.

PROMPT · FULL PROCESSINGSCREEN WITH ANIMATED AGENTCARDS (ANTIGRAVITY)

Build a complete ProcessingScreen with all six AgentCards using hardcoded stub data. The animation timing must be demo-realistic: each card activates at a deliberate offset so the pipeline looks like it's thinking.

FILE: mobile/lib/screens/processing_screen.dart

THE SIX AGENT CARDS — names, stub content, activation offset:

Index 0: "Intent Agent"

content: "Service: AC Repair · Location: G-13 · Time: Tomorrow 10am · Complexity: Intermediate · Confidence: 0.92"

color accent: COBALT

activation offset: 1500ms after screen loads

Index 1: "Discovery Agent"

content: "7 providers found in G-13 · 4 available tomorrow · 1 alternate slot suggested"

color accent: FOREST

activation offset: 3000ms

Index 2: "Ranking Agent"

content: "Ali Plumbing — 1.4km · 4.8 rating · 98% on-time · Inverter AC specialist"

color accent: GOLD

activation offset: 5000ms

Index 3: "Pricing Agent" [NEW — v2]

content: "Estimate PKR 2,260–2,890 · Visit fee + service time + complexity adjustment"

color accent: Color(0xFF92400E) [amber-dark]

activation offset: 6500ms

Index 4: "Booking Agent"

content: "Booking BUL-20260516-9KP3X4 confirmed · Receipt ready"

color accent: CRIMSON

activation offset: 8000ms [only activates after user confirms on PricingScreen]

Index 5: "Follow-up Agent"

content: "Reminder set: 9:30am tomorrow · Rating prompt at 12pm"

color accent: COBALT

activation offset: 9500ms [same caveat as Booking]

ANIMATION SPEC per card (AgentCard widget in mobile/lib/widgets/agent_card.dart):

Pending → Active transition:

- Card background animates from PAPER to WHITE over 200ms.
- Left border appears: 2px GOLD, animate width 0→2px over 200ms.
- Agent name color changes from SLATE to INK.
- Lottie spinner (assets/lottie/spinner.json) appears in content area.

Active → Done transition (triggered after content is ready):

- Lottie spinner fades out over 150ms.
- TypewriterAnimatedText runs on the content string at speed 30ms/char.
- When TypewriterAnimatedText completes: gold checkmark icon fades in top-right.
- Card height animates to accommodate the full content text.

SCREEN FLOW:

When screen loads: start timer sequence for cards 0-3 (index 0, 1, 2, 3).

When Ranking card reaches Done: navigate to PricingScreen (stub).

Cards 4 and 5 (Booking, Follow-up) activate AFTER user returns from PricingScreen with confirm=true.

STATE MANAGEMENT:

Use a ChangeNotifier (ProcessingViewModel) that holds:

List cardStatuses (length 6)

List cardContents (length 6)

bool awaitingPriceConfirm

bool bookingComplete

The ViewModel is provided via Provider at the top of the widget tree.

ASSETS:

Create assets/lottie/ and add a placeholder JSON file (Lottie loading dot animation).

In pubspec.yaml, add:

assets:

- assets/lottie/

After building, run on emulator. Confirm:

- All 6 cards visible in stub pending state.
- Cards animate in sequence with correct offsets.
- TypewriterAnimatedText runs correctly on each card.
- Gold checkmark appears after text completes.
- Screen scrolls if content exceeds viewport.

Commit:

git add mobile/

git commit -m "feat(mobile): HomeScreen + ProcessingScreen with 6 animated stub AgentCards"

git push

End-of-day verification

- flutter analyze shows zero errors.
 - HomeScreen renders: INK AppBar, MicButton with gold fill, subtitle text.
 - Long-press on MicButton shows scale + pulse animation.
 - ProcessingScreen shows 6 cards, all in pending state initially.
 - Cards animate to active then done in the correct order (0→1→2→3).
 - TypewriterAnimatedText runs on each card's content.
 - Gold checkmark appears after text completes.
 - Backend not yet wired — all data is hardcoded stubs.
- *Send Backend the exact JSON shape you expect from /orchestrate as a gist comment. Any change to the shape after today costs you half a day.*

05 · DAY 3 · MAY 16

Day 2 · wire to real backend

By 8pm tonight, a typed Pakistani Urdu sentence produces real Intent + Discovery results on the screen. Voice recording is wired. The first two agent cards show live data. Cards 3-6 still show stubs.

Morning · API service + response models

The Backend teammate has a real /orchestrate endpoint returning live Intent + Discovery data by this morning. Wire the API service, parse the response into Dart models, and replace the stub content with real data on the first two AgentCards. Cards 3-6 still show stubs; the rest come online Day 3.

PROMPT · API SERVICE + DART RESPONSE MODELS (ANTIGRAVITY)

Build the API service and Dart response models that connect the Flutter app to the backend.

FILE: mobile/lib/services/api_service.dart

```
class ApiService {
  static const _baseUrl = String.fromEnvironment('BACKEND_BASE_URL',
    defaultValue: 'https://bulao-backend--as.a.run.app');

  Future orchestrate(String userText) async {
    final uri = Uri.parse('$_baseUrl/orchestrate');
    final response = await http.post(uri,
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({'user_text': userText, 'user_id': 'demo_user_001'}),
    );
    if (response.statusCode != 200) {
      throw ApiException('orchestrate failed: ${response.statusCode}');
    }
    return OrchestrateResponse.fromJson(jsonDecode(response.body));
  }

  Future book(String quotId, String userId) async { ... }

  Future dispute(String bookingId, int rating,
    String complaint, List< checklist) async { ... }

  Future healthCheck() async { ... }
}

class ApiException implements Exception {
  final String message;
  ApiException(this.message);
}
```

RESPONSE MODELS (mobile/lib/models/orchestrate_response.dart):

Dart classes with fromJson factories for every response shape:

```

class IntentResult {
  final String serviceType; // e.g. "ac_technician"
  final String? location;
  final String city;
  final String timeWindow;
  final String urgency;
  final String jobComplexity;
  final String? specializationHint;
  final String genderPreference;
  final double confidence;
  final String? clarificationQuestion;
  final String? rawNotes;
  factory IntentResult.fromJson(Map json) { ... }

  // Display helpers:
  String get serviceLabel => _serviceLabels[serviceType] ?? serviceType;
  String get complexityLabel => { 'basic': 'Basic', 'intermediate': 'Intermediate', 'complex': 'Complex'
}[jobComplexity] ?? jobComplexity;
  String get confidencePercent => '${(confidence * 100).toStringAsFixed(0)}%%';
}

static const _serviceLabels = {
  'plumber': 'Plumber', 'electrician': 'Electrician', 'ac_technician': 'AC Technician',
  'geyser_technician': 'Geyser Technician', 'carpenter': 'Carpenter', 'painter': 'Painter',
  'beautician': 'Beautician', 'tutor': 'Tutor', 'appliance_repair': 'Appliance Repair',
  'gas_leak_specialist': 'Gas Leak Specialist',
};

class ProviderCandidate {
  final String id, name, neighborhood;
  final List specializations;
  final double distanceKm, rating, onTimeScore, riskScore;
  final int completedJobs;
  final String availabilityStatus, nextSlot;
  final int baseVisitFeePkr, ratePerHourPkr;
  factory ProviderCandidate.fromJson(Map json) { ... }
}

class DiscoveryResult {
  final List candidates;
  final List alternates;
  final String? noMatchReason;
  factory DiscoveryResult.fromJson(Map json) { ... }
}

```

```
class RankingResult {
  final String? recommendedId;
  final List topThreeIds;
  final Map factorScores; // nested map
  final String reasoningEnglish;
  final String reasoningUrdu;
  final String? tradeoffs;
  final String confidence; // "high" | "medium" | "low"
  factory RankingResult.fromJson(Map json) { ... }
}

class PriceLineItem {
  final String labelEnglish, labelUrdu;
  final int amountPkr;
  final String kind; // "base" | "addon" | "multiplier" | "discount" | "surge"
  factory PriceLineItem.fromJson(Map json) { ... }
}

class PricingResult {
  final List lineItems;
  final int estimatedTotalPkr;
  final List estimatedRangePkr; // [low, high]
  final String explanationEnglish;
  final String explanationUrdu;
  final String fairnessNote;
  final String quoteId;
  final String expiresAt;
  factory PricingResult.fromJson(Map json) { ... }
}

class OrchestrateResponse {
  final IntentResult intent;
  final DiscoveryResult discovery;
  final RankingResult ranking;
  final PricingResult pricing;
  factory OrchestrateResponse.fromJson(Map json) { ... }
}

class BookingResponse {
  final String bookingId;
  final String confirmationUrdu;
  final String confirmationEnglish;
  final String receiptUrl;
  factory BookingResponse.fromJson(Map json) { ... }
}
```

```
class DisputeResponse {
  final String classification;
  final String resolution;
  final int? refundPkr;
  final String messageUrdu;
  final String messageEnglish;
  final bool escalateToHuman;
  factory DisputeResponse.fromJson(Map json) { ... }
}
```

After building, run:

```
flutter analyze # zero errors
```

```
flutter test # pass
```

Then update ProcessingScreen to call ApiService().orchestrate(userText) when it loads. Show a CircularProgressIndicator while waiting. On success, populate card 0 (Intent) and card 1 (Discovery) with real data. Cards 2-5 still show stubs until Day 3.

Afternoon · voice input + live Intent + Discovery cards

Wire the actual microphone recording so pressing the MicButton on HomeScreen records audio, sends it to Gemini 3 Pro for transcription (STT), and passes the transcript to /orchestrate. By end of day, a spoken Urdu sentence produces real Intent + Discovery results on the screen.

PROMPT · VOICE RECORDING + STT + LIVE DATA ON FIRST TWO CARDS (ANTIGRAVITY)

Wire microphone recording and speech-to-text, then populate the first two AgentCards with live data from the backend.

VOICE INPUT (mobile/lib/widgets/mic_button.dart):

1. Request microphone permission using permission_handler:

```
if (await Permission.microphone.request().isGranted) { startRecording(); }
else { show a SnackBar: "Mic permission zaruri hai" }
```

2. Recording with flutter_sound:

```
FlutterSoundRecorder recorder = FlutterSoundRecorder();
await recorder.openRecorder();
await recorder.startRecorder(toFile: '${tempDir.path}/recording.aac', codec: Codec.aacADTS);
// on long-press end:
final path = await recorder.stopRecorder();
```

3. Speech-to-Text via Gemini API (POST to /stt proxy on the backend, NOT direct to Gemini from Flutter — keep the API key server-side):

```
final transcript = await ApiService().transcribe(audioPath);
// Backend provides POST /stt: accepts multipart form with audio file, returns { "text": "..." }
// If STT fails, show a text input fallback.
```

4. Navigate to ProcessingScreen with transcript:

```
Navigator.pushReplacement(context, MaterialPageRoute(
  builder: (_) => ProcessingScreen(userText: transcript),
));
```

PROCESSING SCREEN with live data:


```

1. On init, call ApiService().orchestrate(widget.userText) with a 45-second timeout.
2. While awaiting: animate card 0 (Intent) as "active" (spinner visible).
3. On response:
  a. Populate card 0 (Intent) content from OrchestrateResponse.intent:
    "${intent.serviceLabel} · ${intent.location ?? 'location TBC'} · ${intent.timeWindow} · ${intent.complexityLabel}
    · Confidence: ${intent.confidencePercent}"
  b. If intent.confidence < 0.7 AND intent.clarificationQuestion != null:
    Show a banner below card 0 in amber: the clarification question as a tappable chip.
    The user can tap to dismiss (proceed anyway) or speak again.
  c. Animate card 1 (Discovery) as active, then populate:
    "${discovery.candidates.length} providers found · ${discovery.alternates.length} alternates"
  d. Animate card 2 (Ranking), then populate from ranking.reasoningUrdu:
    Show first 120 chars of reasoningUrdu + "... [tap to expand]"
  e. Cards 3-5 remain as stubs until Day 3.

4. Error handling:
  - Network timeout (45s): show a bottom sheet: "Request time out ho gaya. Dobra try karein."
  - API error (4xx/5xx): show: "Kuch masla hua. Please try again."
  - No providers found (discovery.candidates.isEmpty): show: discovery.noMatchReason ?? "Koi provider
  available nahi hai"

5. Add a small keyboard icon in the AppBar so user can type instead of speaking (fallback for demo).

Verify on emulator:
  - Type text manually (use the keyboard fallback) to test without recording.
  - Spoken Urdu → transcript appears briefly → cards animate in sequence.
  - Intent card shows service type, location, complexity, and confidence.
  - If confidence < 0.7, the clarification banner appears.
  - Discovery card shows correct provider count.

Commit:
git commit -am "feat(mobile): voice input + live Intent/Discovery cards"
git push

```

End-of-day verification

- Dart models compile with zero errors (flutter analyze).
- ApiService.orchestrate parses the live JSON from the real backend.
- Voice recording starts on long-press and stops on release.
- Transcript is printed to console (debug) before navigating.
- Intent AgentCard shows real service_type, location, complexity, confidence.
- Discovery AgentCard shows the real provider count.
- Clarification banner appears when confidence < 0.7.
- Keyboard fallback allows text-only input.
- Timeout and error handling SnackBars display correctly.

06 · DAY 4 · MAY 17

Day 3 · all six cards + Pricing + Booking success

By 9pm tonight, the full pipeline is end-to-end: voice → intent → discovery → ranking → pricing card → confirm → booking success screen with receipt.

Morning · Ranking card expansion + Pricing Card

Today the Pricing Agent comes online (Backend is wiring it this morning). You build the **Pricing Card** — the key new UI element in v2 that satisfies the rubric's transparent-pricing requirement. The Confirm button is disabled until the user has scrolled the line items, and the 'Why this price?' link expands the Urdu explanation.

PROMPT · RANKING CARD EXPANSION UI + FULL PRICINGCARD WIDGET (ANTIGRAVITY)

Build the full Ranking card expansion UI and the PricingCard widget.

RANKING CARD EXPANSION (in AgentCard widget):

When the user taps "Why?" button on the done Ranking card:

- Expand the card to show a 6-bar horizontal chart.
- Each bar represents one factor score from `ranking.factorScores[recommendedId]`: distance, availability, rating, on_time, specialization, risk
- Bar width proportional to factor score (0.0-1.0), max width = `contentWidth * 0.85`.
- Color: GOLD for top 2 factors, SLATE for others.
- Labels to the left of each bar (use abbreviated names: "Dist.", "Avail.", "Rating", "On-time", "Spec.", "Risk").
- Animate bars growing from 0 to final width over 500ms, staggered 80ms each.
- Below the chart: the "tradeoffs" text from `RankingResult`, in SLATE 12pt italic.
- A "Translate" button toggles between `reasoningUrdu` and `reasoningEnglish`.

FILE: `mobile/lib/widgets/pricing_card.dart`

```
class PricingCard extends StatefulWidget {
  final PricingResult pricing;
  final VoidCallback onConfirm;
  final VoidCallback onCancel;
}
```

VISUAL LAYOUT of PricingCard:

Top: "Price Estimate" in INK Bold 16pt.
 Range: "PKR {estimatedRangePkr[0].toLocale()} – {estimatedRangePkr[1].toLocale()}" in INK Bold 24pt.
 Color GOLD.
 Divider.
 Line items list (scrollable if many):
 For each `lineltem` in `pricing.lineltems`:
 Row: `label_urdu` (left, SLATE 12pt) | `amount_pkr` formatted (right, INK 13pt Bold)
 For discount/surge, amount is colored FOREST (negative) or CRIMSON (surcharge).
 Divider.
 Fairness note: `pricing.fairnessNote` in SLATE italic 11pt.
 "Why this price?" collapsible:
 Collapsed: chevron-down icon + "Why this price?" in GOLD.
 Expanded: `pricing.explanationUrdu` in SLATE 13pt italic.
 Confirm button: "Confirm Booking — PKR {estimatedTotalPkr.toLocale()}"
 - Disabled (grey) until the user has scrolled past all line items (track with `ScrollController`).
 - When enabled: INK background, white text, GOLD border.
 Cancel: "Zyada mehnga? Dusra option" — text button in SLATE.

NAVIGATE TO PRICING CARD:

When the Ranking card (index 2) reaches Done state, navigate to a new screen:
`PricingScreen` (`mobile/lib/screens/pricing_screen.dart`) that wraps `PricingCard`.
`PricingScreen` takes `OrchestrateResponse` as parameter.

ON CONFIRM:

Show a loading indicator. Call `ApiService().book(pricing.quoteId, 'demo_user_001')`.
 On success: activate Booking card (index 4) with real `confirmationUrdu`, then navigate to `BookingSuccessScreen`.
 On `quote_expired` error (backend returns 410): show bottom sheet "Quote expire ho gaya. Naya estimate le rahe hain..." then re-run `/orchestrate`.

LOCALIZED NUMBER FORMAT:

Use `NumberFormat.decimalPattern('en_IN')` for PKR amounts so thousands are comma-separated.

After building, demo path on emulator:

Type → `ProcessingScreen` → all 4 cards animate → `PricingCard` shows line items → tap Confirm → `BookingSuccessScreen`.

Commit: `git commit -am "feat(mobile): PricingCard with line-items + Ranking expansion chart" && git push`

Afternoon · Booking success screen + receipt preview

The Booking Agent writes the confirmation message and generates a receipt PDF. The `BookingSuccessScreen` shows the WhatsApp-style confirmation bubble and a receipt preview that the user can tap to open the full PDF. This screen is a hero asset for the demo; it needs to look genuinely professional.

PROMPT · BOOKINGSUCCESSSCREEN + RECEIPT PDF PREVIEW (ANTIGRAVITY)

Build the `BookingSuccessScreen` and activate the Booking + Follow-up agent cards.

FILE: `mobile/lib/screens/booking_success_screen.dart`

Takes: `BookingResponse` `booking`, `OrchestrateResponse` `orchestrate`

SCREEN LAYOUT (portrait, white background):

Top 35% of screen: success hero area.

- Background: subtle gold gradient (#b8860b → #d4a017), full width.
- Center: white checkmark icon in a circle (80px), with a subtle drop shadow.
- Below checkmark: "Booking Confirm!" in white Bold 22pt.
- Below that: booking.bookingId in white 13pt monospace.

Below hero, scrollable content area:

1. CONFIRMATION BUBBLE (WhatsApp-style):

- Rounded rectangle, CREAM background, GOLD left border 2px.
- "Bulao" label in GOLD Bold 11pt, top-left.
- booking.confirmationUrdu in INK 14pt.
- booking.confirmationEnglish in SLATE 12pt.
- Timestamp (current time) bottom-right, SLATE 11pt.

2. RECEIPT PREVIEW CARD:

- Rounded rectangle, white background, 1px RULE border.
- PDF icon (red) + "Receipt.pdf" in INK Bold 13pt.
- booking.bookingId + date in SLATE 11pt.
- "Tap to open" in GOLD 12pt.
- onTap: open receiptUrl in the device browser.

3. WHAT HAPPENS NEXT section:

- Title: "Aage kya hoga?" in INK Bold 16pt.
- Three timeline items (vertical line with dots):
 1. "Kal subah 9:30 baje" — "Ali pohanch jayenge" (FOREST dot)
 2. "Service complete hone ke baad" — "Aap rating de saktay hain" (GOLD dot)
 3. "Koi masla ho toh" — "Directly report karein" (CRIMSON dot)

4. DONE BUTTON at bottom:

- "Done" in INK Bold.
- Navigates back to HomeScreen (pop to root).

ALSO — after BookingSuccessScreen is shown, go back to ProcessingScreen and activate Booking card (index 4) and Follow-up card (index 5) with their respective content. Trigger the animation sequence for both cards.

Commit: git commit -am "feat(mobile): BookingSuccessScreen + receipt preview" && git push

End-of-day verification

- PricingCard shows all line items from real backend data.
- Line items with negative amounts (discounts) shown in FOREST green.
- Confirm button is disabled until user scrolls past all line items.
- After confirm: Booking + Follow-up AgentCards activate on ProcessingScreen.
- BookingSuccessScreen shows the hero checkmark + gold gradient.
- WhatsApp-style confirmation bubble shows real Urdu text from backend.
- Receipt preview card taps to open the PDF URL.

- 'What happens next' timeline renders correctly.
- Six-factor bar chart on Ranking card expands on 'Why?' tap.
- Translate button toggles between Urdu and English reasoning.

07 · DAY 5 · MAY 18

Day 4 · lifecycle + dispute + edge cases

The dispute flow is 15%% of the rubric. Build it as if a real user's refund depends on it — because at finals, a judge will tap 2 stars and wait.

Morning · LifecycleScreen (en_route → completed)

The service-quality lifecycle is what proves the full end-to-end flow. The demo walks through the status transitions live on screen. Build the LifecycleScreen with the four-stage tracker and simulated status progression (since we don't have a real provider on the street).

PROMPT · LIFECYCLEScreens WITH SERVICE STATUS TRACKER (ANTIGRAVITY)

Build the LifecycleScreen that tracks a booking's service-quality lifecycle.

FILE: mobile/lib/screens/lifecycle_screen.dart

Takes: String bookingId, String providerName, String scheduledTime

LIFECYCLE STAGES (in order):

1. confirmed — "Booking Confirmed"
2. en_route — "Ali aapke taraf aa rahe hain"
3. arrived — "Ali pohanch gaye"
4. in_progress — "Kaam shuru ho gaya"
5. completed — "Kaam mukammal!"

SCREEN LAYOUT:

AppBar: provider name + "· En Route" (updates with current stage).

PROGRESS TRACKER (top 45% of screen):

- Horizontal stepper with 5 nodes, connected by a line.
- Completed nodes: filled GOLD circle, white checkmark.
- Current node: filled GOLD circle, white animated pulse ring.
- Future nodes: SLATE outline circle.
- Below each node: short label (2-3 words) in SLATE 10pt.
- Line color: GOLD for completed sections, RULE for future.

PROVIDER CARD (middle):

- Provider avatar placeholder (colored circle with initial letter).
- Provider name in INK Bold 16pt.
- Stars row: rating e.g. ★★★★★ in GOLD.
- "1.4 km away" in SLATE.
- Phone button (taps to hypothetically call — show a "Feature coming soon" toast).

STATUS MESSAGE (below provider card):

- Big status text matching current stage in INK Bold 18pt.
- Urdu sub-text: stage-specific message (see above).

COMPLETION — when status == completed:

- Status area replaced by: "Kaam kaisa raha?" heading.
- 5-star rating widget (tappable stars, filled GOLD on tap).
- Submit button: "Rating submit karein".
- On submit: if rating < 3, navigate to DisputeScreen. If >= 3, show "Shukriya! Aapka rating save ho gaya." toast and navigate back to HomeScreen.

SIMULATION (for demo):

Add a hidden long-press on the progress tracker that advances to the next stage.

This is the demo control: presenter long-presses the tracker to advance.

For the REAL system, status would be polled via GET /booking/{id}/status.

In demo: simulate en_route → arrived → in_progress → completed with a 1-second animation between stages when the long-press fires.

Commit: git commit -am "feat(mobile): LifecycleScreen with service status tracker" && git push

Late morning · DisputeScreen + resolution UI

The dispute flow is 15% of the rubric score. The DisputeScreen must look polished enough that a judge believes a real user would trust it with a refund dispute. The key design goal: every step feels reassuring, not aggressive or confrontational.

PROMPT · DISPUTESCREEN + RESOLUTION CHAT UI (ANTIGRAVITY)

Build the DisputeScreen and the resolution UI it leads to.

FILE: mobile/lib/screens/dispute_screen.dart

Takes: String bookingId, String providerName, int rating

THE DISPUTE SCREEN has THREE STEPS (shown as a 3-step progress indicator at the top):

STEP 1 — Service checklist (what was and wasn't done):

Heading: "Kya kaam hua?" in INK Bold 20pt.

Sub: "Jo kaam hua use check karein" in SLATE 14pt.

A list of 5 checklist items (specific to service type — use hardcoded AC repair for demo):

- "Filter clean kiya gaya" (Filter cleaned)
- "Gas refill kiya gaya" (Gas refilled)
- "Outdoor unit check kiya" (Outdoor unit checked)
- "Test cooling kiya" (Test cooling done)
- "Koi nuksaan nahi hua" (No damage done)

Each is a tappable row: tap toggles ■/■.

"Aage" (Next) button enabled after user checks at least one item.

STEP 2 — Describe the issue:

Heading: "Kya masla hua?" in INK Bold 20pt.

Text field (multiline, 4 lines): "Apna masla yahan likhen..."

Pre-filled tags (tappable chips, user can tap to add text):

"Kaam adhura raha" · "Zyada paise liye" · "Behavior theek nahi tha" · "Problem wapas aa gayi"

"Submit" button.

STEP 3 — Resolution (shows after API call returns):

The DisputeResponse from POST /dispute.

If `escalate_to_human == true`:

Show: hourglass icon + "Aapka complaint review ho raha hai" (SLATE).

Sub: "24 ghante mein response milega."

Else:

Heading: "Aapka masla resolve ho gaya" in FOREST Bold 18pt.

Green checkmark icon.

Resolution message: `disputeResponse.messageUrdu` in INK 14pt (inside a CREAM card).

Refund amount (if `resolution == partial_refund` or `full_refund`):

"PKR {`refundPkr`} wapaa aayenge — 3-5 business days mein" in GOLD Bold 16pt.

Re-service option (if `resolution == re_service`):

"Free re-service schedule karein" button.

"Done" button → HomeScreen.

API CALL:

On Step 2 submit: call `ApiService().dispute(bookingId, rating, complaint, checklist)`.

Show a loading animation for 2-3 seconds (make the "thinking" visible — this is the agent working, which counts for the Agentic Reasoning rubric score).

When result arrives: animate to Step 3.

ERROR:

If the call fails: "Kuch masla hua. Dohara try karein." with a Retry button.

Commit: `git commit -am "feat(mobile): DisputeScreen 3-step flow + resolution UI" && git push`

Afternoon · edge case UIs + low-confidence clarification

Judges will probe edge cases at finals. Build the three most likely failure-mode UIs so they never see a raw error: no providers found, quote expired, and the clarification chip for low-confidence intent. These are small but high-stakes.

PROMPT · EDGE CASE SCREENS: NO MATCH, QUOTE EXPIRED, CLARIFICATION (ANTIGRAVITY)

Build three edge-case UI states that gracefully handle failure modes judges will test.

EDGE CASE 1 — No providers found (`discovery.candidates.isEmpty`):

In ProcessingScreen, after Discovery card activates:

If `discovery.noMatchReason` is not null AND `candidates.isEmpty`:

Replace Discovery card content with:

CRIMSON border instead of FOREST.

Text: "Koi provider available nahi" in CRIMSON Bold.

Sub-text: `discovery.noMatchReason` in SLATE 12pt.

Below: "Nazdiki ilaaka try karein?" with a "Try nearby" button that changes location filter.

Halt the pipeline. Cards 2-5 stay pending.

Show a bottom sheet: "Filhal koi provider available nahi hai waqt. Hum notify karenge jab koi mile."

"OK" dismisses to HomeScreen.

EDGE CASE 2 — Quote expired (`ApiService.book` returns 410 status):

The PricingCard's Confirm button handler catches this:

Show a bottom sheet with an hourglass animation:

"Quote expire ho gaya (15 minute guzar gaye)"

"Naya estimate le rahe hain..."

Auto-re-run ApiService().orchestrate(originalUserText) after 1.5 seconds.

On new response: update PricingCard with new pricing.

The quote_id in the Confirm button is updated automatically.

EDGE CASE 3 — Low confidence clarification chip:

In ProcessingScreen, when intent.confidence < 0.7 AND intent.clarificationQuestion != null:

Show a yellow banner card below the Intent AgentCard:

Title: "Thodi wazahat chahiye" (Needs clarification) in Color(0xFF92400E) Bold.

Text: intent.clarificationQuestion in INK 14pt.

Two tappable answer chips (pre-built from known neighborhoods):

e.g. if clarification topic is location, show: "G-13", "F-10", "G-11", "Saddar" as chips.

OR: a small text field for free-form answer.

"Jawab dein" (Answer) button.

When user answers: re-run orchestrate with original_text + " [clarification: \${answer}]".

On new response: update Intent card, hide the banner.

Commit: git commit -am "feat(mobile): no-match, quote-expired, clarification edge cases" && git push

End-of-day verification

- LifecycleScreen shows 5-stage horizontal stepper with gold/slate styling.
- Long-press on tracker advances to next stage with animation.
- Rating widget tappable; rating < 3 navigates to DisputeScreen.
- DisputeScreen Step 1 shows service checklist with toggleable items.
- DisputeScreen Step 2 has multiline input + pre-filled issue chips.
- Dispute API called on submit; 2-3 second thinking animation visible.
- DisputeScreen Step 3 shows resolution with refund amount in GOLD.
- No-match state shows friendly Urdu message, not a raw error.
- Quote-expired bottom sheet auto-re-runs orchestrate after 1.5 seconds.
- Clarification banner appears with answer chips when confidence < 0.7.

08 · DAY 6 · MAY 19

Day 5 · polish, APK, screen recordings

No new features. Polish the visual treatment, sign the APK, and deliver three screen recordings to the Infra teammate for the video edit.

Morning · full demo flow polish pass

No new features. Read the demo storyboard in the Product team's Role Pack Appendix C. Walk every beat and fix any rough edges. Judges watch the demo on a phone screen — anything that looks unfinished will look unfinished.

PROMPT · POLISH PASS: ANIMATIONS, TEXT, LAYOUT (ANTIGRAVITY)

Conduct a visual polish pass on the full Bulao Flutter app. Do not add features. Fix things that look unfinished.

CHECKLIST (run through this in order on the emulator):

1. HOME SCREEN

- AppBar background is INK (#1A1D29). Check.
- MicButton pulse animation: is the GOLD consistent? Check the exact hex.
- "Kya kaam hai aaj?" is in SLATE, centered. Check.
- On landscape rotation, does the layout break? Fix if so (use LayoutBuilder).

2. PROCESSING SCREEN

- Cards are evenly spaced with 12px gap between.
- Each card has consistent 16px horizontal padding.
- The TypewriterAnimatedText does not overflow on long content (clip + fade).
- The "Why?" button on the Ranking card has a GOLD border, not default Flutter blue.
- Agent card names are in the correct color (COBALT/FOREST/GOLD/amber/CRIMSON/COBALT).
- Gold checkmark icon: is it the correct Assets icon, not an emoji?

3. PRICING CARD

- Line items with discount show negative amount in FOREST green.
- Surge amounts show in CRIMSON.
- PKR amounts have comma separators (1,920 not 1920).
- "Why this price?" expands smoothly with a Curves.easeOut animation.
- Fairness note is visible and styled in SLATE italic.

4. BOOKING SUCCESS SCREEN

- Gold gradient is smooth, not banded.
- Confirmation bubble has the correct CREAM background + GOLD left border.
- Receipt card has a file icon, not a placeholder.
- "What happens next" timeline: vertical line connects all three dots.

5. LIFECYCLE SCREEN

- Progress stepper scales to smaller phones (use FittedBox on labels).
- Pulse animation on current stage node: should pulse every 1.5s, not continuously fast.

6. DISPUTE SCREEN

- Progress steps at the top match the current step.
- Chips in Step 2 wrap correctly on narrow screens.
- Refund amount appears only when refundPkr > 0.

7. GLOBAL

- No debug banner (flutter run --release, or add debugShowCheckedModeBanner: false).
- No overflow yellow/black stripes anywhere.
- All text is in DM Sans (check with flutter inspector: verify fontFamily).
- No raw error messages visible to user anywhere (all errors are handled).

Fix each issue as you go. Report what was changed. Do not add new functionality.

Late morning · sign and build the APK

The Infra teammate needs a signed APK to host for judges to download. Today you generate the release keystore, sign the APK, upload it, and send the download link to Infra.

PROMPT · SIGN AND BUILD THE RELEASE APK (ANTIGRAVITY)

Generate a signed release APK for the Bulao Flutter app and prepare it for distribution.

1. Generate a release keystore (run this yourself in Terminal, NOT in Antigravity — protect your key):

```
keytool -genkey -v \
-keystore ~/bulao-release-key.jks \
-keyalg RSA -keysize 2048 -validity 10000 \
-alias bulao-key
```

Answer the prompts. Remember the keystore password.

DO NOT commit the .jks file to git.

2. Create mobile/android/key.properties (DO NOT commit this file either):

```
storePassword=
keyPassword=
keyAlias=bulao-key
storeFile=/Users//bulao-release-key.jks
```

3. Update mobile/android/app/build.gradle to read key.properties for signingConfigs.

Antigravity prompt (paste this):

Open mobile/android/app/build.gradle. Add a signingConfigs block that reads from the key.properties file at the project root. Configure the release build type to use this signingConfig. Then run:

```
flutter build apk --release --dart-define=BACKEND_BASE_URL=https://bulao-backend--as.a.run.app
```

Report the output file path and size.

4. After the build:

- Verify the APK is at build/app/outputs/flutter-apk/app-release.apk.
- Confirm size is < 50MB.
- Upload to Google Drive shared folder (or Firebase App Distribution if set up by Infra).
- Send the download link to Infra teammate via WhatsApp.
- Paste the download link into README.md under the APK section.

5. Test install the APK on a real Android device:

```
adb install build/app/outputs/flutter-apk/app-release.apk
```

Run through the full demo flow once.

Commit:

```
git add mobile/android/app/build.gradle
git commit -m "release: configure signing for release APK"
git push
```

(Do NOT commit key.properties or .jks)

Afternoon · demo screen recordings for Infra

The Infra teammate edits the demo video from your screen recordings. You record three angles today on a real Android device using ADB screen recording and/or a physical camera over-the-shoulder.

PROMPT · SCREEN RECORDING CHECKLIST (DO IN PERSON, NOT IN ANTIGRAVITY)

Record the demo screen captures for the Infra teammate to use in the final video edit.

DEVICE PREP:

- Use a real Android device (not an emulator).
- The signed release APK must be installed.
- Phone is on Do Not Disturb. Notifications silenced.
- Battery $\geq$ 80%.
- Backend Cloud Run service confirmed reachable (ask Infra).
- Wi-Fi stable, OR use team phone hotspot.
- DEMO_MODE: keep false for the real run; keep a DEMO_MODE=true APK build as fallback.

RECORDINGS TO MAKE:

Recording 1 — ADB screen mirroring (clean, lossless):

Connect phone via USB. Run:

```
adb exec-out screenrecord --output-format=h264 - | ffplay -
```

This mirrors to your laptop. Use OBS to capture OBS output as MP4.

Do the FULL demo run from HomeScreen → confirm booking → receipt.

Target duration: 3:30 (excluding closing card).

Recording 2 — Over-the-shoulder phone camera:

A second person holds a camera over the presenter's shoulder pointing at the phone.

This is the "human feels" shot — a real hand, a real phone.

Record the key beats: press-and-hold mic, cards animating, Pricing Card, confirm.

30-60 seconds of clean footage.

Recording 3 — Dispute path only:

Start on the LifecycleScreen in "completed" state.

Give rating 2 stars.

Go through DisputeScreen all three steps.

See the refund resolution.

40 seconds.

AFTER RECORDING:

- AirDrop / USB transfer all recordings to the Infra teammate.
- Share the demo voiceover script (docs/demo-voiceover.md) from the product team.
- Confirm Infra has docs/demo-assets/closing_card.mp4 from the Product lead.

FALLBACK PLAN:

If voice input fails on the real device during the shoot:

- Switch to the keyboard icon in the AppBar — type the Urdu text.
- The voiceover script doesn't need to change.
- Do NOT cancel the shoot; text-input is acceptable for the demo.

End-of-day verification

- Zero overflow warnings in the complete demo flow on the emulator.
- All text rendered in DM Sans (debugShowCheckedModeBanner: false set).
- Signed release APK built and installs cleanly on a real Android device.
- APK size < 50MB.
- Download link sent to Infra teammate.
- Three screen recordings delivered to Infra.
- README updated with APK download link.
- All commits pushed with clean history.

APPENDIX A

The API contract — exactly what the backend sends.

This is the contract between you and the Backend Agents + Workflows leads. Do not deviate from the field names, nesting, or types shown here. After Day 1 evening this contract is frozen — any change requires a WhatsApp team decision.

```
/orchestrate
POST body: { "user_text": "...", "user_id": "..." }
Response 200:
{
  "intent": {
    "service_type": "ac_technician",
    "location": "G-13", "city": "Islamabad",
    "time_window": "tomorrow_morning",
    "urgency": "normal",
    "job_complexity": "intermediate",
    "specialization_hint": "inverter_ac",
    "gender_preference": "any",
    "confidence": 0.92,
    "clarification_question": null,
    "raw_notes": null
  },
  "discovery": {
    "candidates": [
      {
        "id": "prov_042", "name": "Ali Plumbing Works",
        "neighborhood": "G-13", "distance_km": 1.4,
        "rating": 4.8, "completed_jobs": 127,
        "on_time_score": 0.96, "cancellation_rate": 0.03,
        "risk_score": 0.08, "current_workload": 0.4,
        "specializations": ["inverter_ac", "split_ac"],
        "availability_status": "available_now",
        "next_slot": "2026-05-17T10:00:00+05:00",
        "base_visit_fee_pkr": 500, "rate_per_hour_pkr": 1200,
        "gender": "male", "years_experience": 12,
        "phone_masked": "+923XX-XXX-1234", "verified": true
      }
    ]
  }
}
```

```

    }
  ],
  "alternates": [],
  "no_match_reason": null
},
"ranking": {
  "recommended_id": "prov_042",
  "top_three_ids": ["prov_042", "prov_017", "prov_089"],
  "factor_scores": {
    "prov_042": {
      "distance": 0.82, "availability": 1.0, "rating": 0.96,
      "on_time": 0.98, "specialization": 1.0, "risk": 0.92,
      "total": 0.94
    }
  },
  "reasoning_english": "Ali Plumbing is the strongest match...",
  "reasoning_urdu": "Ali Plumbing sab se behtar hai – sirf 1.4 km...",
  "tradeoffs": "Rashid Plumbing is 15%% cheaper but 4.5 rated.",
  "confidence": "high"
},
"pricing": {
  "line_items": [
    {
      "label_english": "Visit fee", "label_urdu": "Visit fee",
      "amount_pkr": 500, "kind": "base",
    },
    {
      "label_english": "Service time (1.5h)",
      "label_urdu": "Service time (1.5 ghante)",
      "amount_pkr": 1800, "kind": "base",
    },
    {
      "label_english": "Complexity (inverter AC)",
      "label_urdu": "Complexity (inverter AC)",
      "amount_pkr": 575, "kind": "multiplier",
    },

    {
      "label_english": "Welcome discount",
      "label_urdu": "Welcome discount",
      "amount_pkr": -200, "kind": "discount"
    }
  ],
  "estimated_total_pkr": 2675,
  "estimated_range_pkr": [2260, 2890],
  "explanation_english": "Your AC repair estimate is PKR 2,260-2,890...",
  "explanation_urdu": "Aapka AC repair ka estimate PKR 2,260-2,890 hai...",
  "fairness_note": "Provider receives PKR 1,920 after platform fee.",
  "quote_id": "QT-BUL2026-A8K3M2",
  "expires_at": "2026-05-16T14:45:00+05:00"
}
}

/book
POST body: { "quote_id": "QT-...", "user_id": "..." }
Response 200:
{
  "booking_id": "BUL-20260516-9KP3X4",
  "confirmation_urdu": "Assalam-o-Alaikum, aapka AC repair booking confirm ho gaya...",
  "confirmation_english": "Hi, your AC repair booking is confirmed...",
  "receipt_url": "https://storage.googleapis.com/bulao-receipts/BUL-...pdf"
}
Response 410 (quote expired):
{ "error": "quote_expired", "message": "Quote QT-... has expired. Please re-run /orchestrate." }

/dispute
POST body: { "booking_id": "...", "rating": 2, "complaint": "...", "checklist": [...] }
Response 200:
{

```

```
"classification": "quality_complaint",
"resolution": "partial_refund",
"refund_pkr": 1250,
"message_urdu": "Maafi mangte hain ke service achi nahi rahi...",
"message_english": "We're sorry the service didn't meet expectations...",
"escalate_to_human": false
}
```

/health

```
GET → { "status": "ok", "version": "1.0.0" }
```

/stt (speech-to-text proxy)

```
POST multipart form: audio file (AAC)
```

```
Response: { "text": "Mujhe G-13 mein kal subah AC theek karwana hai" }
```


APPENDIX B

Design tokens — every color, font, and spacing value.

Copy `app_theme.dart` from these specs. Every screen uses these tokens — never raw hex values in widget code. Consistent tokens mean consistent UI.

```
// mobile/lib/theme/app_theme.dart – complete token set

// Colors
static const Color ink          = Color(0xFF1A1D29); // primary text, AppBar BG
static const Color gold        = Color(0xFFB8860B); // accent, borders, checkmarks
static const Color goldLight   = Color(0xFFF5EDD0); // card hover, chip BG
static const Color goldDark    = Color(0xFF8B6914); // darker gold for contrast
static const Color cream       = Color(0xFFFEF9E7); // confirmation bubble BG
static const Color paper       = Color(0xFFFAFAF7); // screen background
static const Color slate       = Color(0xFF6B7280); // secondary text
static const Color rule        = Color(0xFFE5E7EB); // dividers, light borders
static const Color mute        = Color(0xFF9CA3AF); // placeholder text, disabled
static const Color cobalt      = Color(0xFF1E40AF); // Intent card accent
static const Color forest      = Color(0xFF166534); // Discovery card, positive amounts
static const Color crimson     = Color(0xFF991B1B); // Booking card, negative (surcharge)
static const Color amberDark   = Color(0xFF92400E); // Pricing card accent
static const Color white       = Color(0xFFFFFFFF);

// Typography – DM Sans everywhere
static const TextStyle h1 = TextStyle(fontFamily: 'DMSans', fontSize: 28,
  fontWeight: FontWeight.bold, color: ink);
static const TextStyle h2 = TextStyle(fontFamily: 'DMSans', fontSize: 20,
  fontWeight: FontWeight.bold, color: ink);
static const TextStyle body = TextStyle(fontFamily: 'DMSans', fontSize: 15,
  fontWeight: FontWeight.normal, color: ink);
static const TextStyle bodySmall = TextStyle(fontFamily: 'DMSans', fontSize: 13,
  fontWeight: FontWeight.normal, color: slate);
static const TextStyle caption = TextStyle(fontFamily: 'DMSans', fontSize: 11,
```

```

    fontWeight: FontWeight.normal, color: slate);
static const TextStyle labelBold = TextStyle(fontFamily: 'DMSans', fontSize: 12,
    fontWeight: FontWeight.bold, color: ink, letterSpacing: 0.8);

// Spacing (all in logical pixels)
static const double spaceXS = 4;
static const double spaceSM = 8;
static const double spaceMD = 12;
static const double spaceLG = 16;
static const double spaceXL = 24;

// Border radii
static const double radiusCard = 12;
static const double radiusChip = 20;
static const double radiusButton = 8;

// Card elevation / shadow
static BoxDecoration cardDecoration = BoxDecoration(
    color: white,
    borderRadius: BorderRadius.circular(radiusCard),
    boxShadow: [BoxShadow(color: Colors.black12, blurRadius: 8, offset: Offset(0, 2))],
);

// Agent card GOLD left-border decoration
static BoxDecoration agentCardActive = BoxDecoration(
    color: white,
    border: Border(left: BorderSide(color: gold, width: 2)),
    borderRadius: BorderRadius.circular(radiusCard),

    boxShadow: [BoxShadow(color: Colors.black08, blurRadius: 6, offset: Offset(0, 1))],
);

// Button styles
static ButtonStyle primaryButton = ElevatedButton.styleFrom(
    backgroundColor: ink, foregroundColor: white,
    textStyle: const TextStyle(fontFamily: 'DMSans', fontSize: 15, fontWeight: FontWeight.bold),
    padding: EdgeInsets.symmetric(horizontal: 24, vertical: 14),
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(radiusButton)),
);
static ButtonStyle goldOutlineButton = OutlinedButton.styleFrom(
    foregroundColor: gold,
    side: BorderSide(color: gold, width: 1.5),
    textStyle: const TextStyle(fontFamily: 'DMSans', fontSize: 14),
    padding: EdgeInsets.symmetric(horizontal: 20, vertical: 12),
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(radiusButton)),
);

```

APPENDIX C

Screen map — widget hierarchy for all 6 screens.

The complete widget tree for every screen. Use this when you give Antigravity a build prompt — paste the relevant screen spec and the agent will reproduce the exact hierarchy without guessing.

Bulao Flutter App – Screen Map

HomeScreen (home_screen.dart)

Scaffold

AppBar: "Bulao" – INK bg, GOLD title

Body: Center > Column

Text "Kya kaam hai aaj?" – SLATE h2

SizedBox(height: 40)

MicButton (widget) – 140px diameter

SizedBox(height: 24)

Text "Tap and hold to speak" – SLATE caption

SizedBox(height: 16)

IconButton(keyboard) → text input mode overlay

ProcessingScreen (processing_screen.dart)

ChangeNotifierProvider<ProcessingViewModel>

Scaffold

AppBar: "Processing..." or service label

Body: SingleChildScrollView > Padding > Column

Text (userText preview) – SLATE 12pt

SizedBox(height: 12)

AgentCard × 6 (agent_card.dart) – see widget spec below

SizedBox(height: 24)

PricingScreen (pricing_screen.dart)

Scaffold

AppBar: "Price Estimate"

Body: SingleChildScrollView

PricingCard (widget) – see widget spec below

```

BookingSuccessScreen (booking_success_screen.dart)
  Scaffold (no AppBar – full screen)
    Column
      HeroArea (35% height) – gold gradient, checkmark, booking_id
      Expanded > SingleChildScrollView > Column
        ConfirmationBubble (CREAM bg, GOLD border)
        ReceiptPreviewCard (white bg, PDF icon, tap to open)
        WhatNextTimeline (3 steps)
      DoneButton

LifecycleScreen (lifecycle_screen.dart)
  Scaffold
    AppBar: providerName + current stage
    Body: Column
      ProgressStepper (5 nodes – confirmed/en_route/arrived/in_progress/completed)
      SizedBox
      ProviderCard (avatar, name, stars, distance, phone button)
      StatusMessageCard
      IF completed: RatingWidget → if < 3 → navigate to DisputeScreen

DisputeScreen (dispute_screen.dart)
  Scaffold
    AppBar: "Masla Report Karein" – CRIMSON accent
    Body: Stepper (3 steps)
      Step 1: ServiceChecklistStep (5 toggle items)
      Step 2: IssueDescriptionStep (text field + chips)
      Step 3: ResolutionStep (shows DisputeResponse)

AGENT CARD WIDGET (agent_card.dart):
  AgentCard

  AnimatedContainer (height, bg color, border)
    Padding
      Row (agent name + status icon)
      SizedBox(height: 8)
      IF active: Lottie(spinner) + status text
      IF done: TypewriterAnimatedText(content) + optional expand button
      IF pending: nothing (collapsed card)

PRICING CARD WIDGET (pricing_card.dart):
  Card
    Column
      Text "Price Estimate"
      Text "PKR X,XXX – X,XXX" – GOLD Bold 24pt
      Divider
      ListView.builder(lineItems) (scroll-tracked)
        PriceLineItemRow (label_urdu | amount_pkr)
      Divider
      Text fairnessNote – SLATE italic 11pt
      WhyThisPrice (collapsible)
      SizedBox
      ConfirmButton (disabled until scroll complete)
      CancelButton

```

APPENDIX D

Agent card timing — activation offsets for the demo.

Buffer reveals to match these offsets even when the API responds faster. The deliberate pacing is what makes the pipeline feel like it is thinking, not printing. That is the agentic moneyshot.

Agent card reveal timing for the demo pipeline.

These offsets are from the moment ProcessingScreen loads.
They simulate realistic LLM latency without actually waiting for slow calls.

Card 0: Intent Agent
Activate at: 1.5s
Content ready at: 3.0s
Note: if real call returns faster, hold until 3.0s (buffer for drama)

Card 1: Discovery Agent
Activate at: 3.0s
Content ready at: 4.5s

Card 2: Ranking Agent
Activate at: 4.5s
Content ready at: 6.5s
Note: after done, auto-navigate to PricingScreen after 1.0s delay

Card 3: Pricing Agent
Activate at: 6.5s
Content ready at: 8.0s
Note: PricingScreen is shown SIMULTANEOUSLY; this card shows pricing in the background. When user confirms on PricingScreen, cards 4+5 activate.

Card 4: Booking Agent
Activate at: user confirms pricing + 1.0s
Content ready at: user confirms pricing + 3.5s

Card 5: Follow-up Agent
Activate at: user confirms pricing + 3.5s
Content ready at: user confirms pricing + 5.0s

TOTAL DEMO VISIBLE PIPELINE TIME (from screen load to all cards done): ~13 seconds
On a real end-to-end call, this should be 15-25 seconds. Buffer by holding reveals until the scheduled offsets even if the API returns faster.